

Cahier des Charges (CDC) — SAAIA Agent IA RAG

Solution On-Prem (Industrie / Entreprise)

Version : v2.8

Date : 2026-02-24

Auteur : Maxime Birchler + Assistant IA

0. Résumé exécutif

SAAIA est un assistant IA RAG 100% on-prem destiné aux environnements industriels/entreprise. Il ingère automatiquement un dossier de documents (PDF principalement), indexe le contenu et répond aux questions en citant précisément les sources (document, page, extrait).

Décisions v2.8 (verrouillées) :

- Serveur sans LLM : aucune génération côté serveur ; le serveur fournit ingestion, RAG retrieval, chat-store, sécurité, observabilité.
- Client WinUI : LLM local obligatoire et embarqué (llama.cpp server OpenAI-compatible), avec téléchargement automatique du modèle.
- CPU garanti ; accélération GPU automatique si supportée (sans choix utilisateur).
- Cible runtime : .NET 8 (LTS) pour le serveur, le client et les contrats partagés.
- Licensing on-prem : licence entreprise (expiration + maxSeats) + clés seat par poste (activation/binding serveur).
- Catégories dynamiques : ajout/suppression/renommage de dossiers dans documents sans recoder.
- Deux installateurs : Server Linux (pack + scripts) et Client Windows (EXE/MSIX).

1. Objectifs produit

1.1 Objectif principal

Permettre aux utilisateurs de retrouver et exploiter rapidement des informations dans de nombreux documents techniques, tout en gardant les données confidentielles et locales.

1.2 Objectifs secondaires

- Réduire le temps de recherche documentaire.
- Standardiser les réponses (sourcing, structure, transparence).
- Améliorer la qualité d'assistance : clarification, plan, prochaines étapes, mémoire conversationnelle.
- Industrialiser : sécurité, observabilité, packaging, runbook.

2. Portée et contraintes

2.1 Portée (in)

- Ingestion automatique de documents (PDF prioritaire).
- Indexation vectorielle + métadonnées.
- Recherche (retrieval) exposée via API serveur.
- Client WinUI : interface chat, sources cliquables, multi-turn, streaming local.
- LLM local (obligatoire) piloté par le client.

2.2 Hors-périmètre (out)

- Accès Internet pour enrichissement externe (interdit par design).
- Agents tools externes (Jira, email, etc.) - versions futures.
- SSO/AD - option future.

2.3 Contraintes fortes

- On-prem : aucun flux de données vers un service tiers.

- Serveur sans LLM : pas de génération serveur, pas de modèle serveur.
- LLM client obligatoire : installé/configuré automatiquement, sans dépendances externes imposées.
- Mode dégradé : si LLM indisponible, l'utilisateur peut au minimum rechercher, consulter les sources, exporter un diagnostic.

3. Architecture cible

3.1 Vue globale

- Server RAG Core (Linux recommandé) : ingestion (Watcher/Scanner/Worker), Postgres, Qdrant, TEI, API Backend .NET.
- Client Chat Agent (Windows WinUI) : UI chat + orchestration assistant + LLM local OpenAI-compatible embarqué + mémoire chat serveur.

3.2 Principes d'architecture

- Le serveur est stateless pour la génération (pas de LLM).
- Le client est responsable de : intention, plan de retrieval, génération de réponse, streaming UI.
- Le serveur est responsable de : RAG retrieval, ingestion/cohérence index, sécurité/licensing, audit/observabilité, chat-store.

4. Profils de déploiement

4.1 Profil Serveur - Linux Compose

- OS : Linux (VM ou bare metal).
- Runtime : Docker Engine + Docker Compose.
- Services : backend (.NET 8), postgres, qdrant, tei-embeddings (+ OTel optionnel).
- Aucun service LLM.

4.2 Profil Client - Windows

- OS : Windows 10/11.
- App : WinUI 3 (EXE recommandé ; MSIX possible).
- Inclus : binaire llama.cpp server (OpenAI-compatible) embarqué.
- Modèles : téléchargés au premier run (avec reprise + checksum).

5. Données et modèles

5.1 Entités principales (Postgres)

- tenants
- licenses (licence entreprise signée : expireAt, maxSeats, features)
- seats (seatKey par poste + binding device)
- api_keys (admin)
- documents, ingestion_jobs, audit_events
- chat_sessions, chat_messages

5.2 Catégories (dossiers)

La catégorie d'un document est dérivée du chemin dans le répertoire documents (par défaut : premier niveau de dossier). Le système doit permettre d'ajouter/supprimer/renommer les dossiers sans recoder.

- Ajouter un dossier = nouvelle catégorie.
- Renommer un dossier = catégorie mise à jour après scan (mouvement traité comme delete+upsert).
- Supprimer un dossier = documents marqués missing_since, purge contrôlée (anti-wipe).
- Le client liste les catégories via GET /rag/categories (dynamique).

6. API serveur

6.1 Auth + Licensing

Headers :

- X-API-Key: <seatKey> (poste) ou <adminKey>
- X-User-Id: <GUID client> (chat-store)

Règles :

- Licence expirée : blocage global des routes utilisateur (403).
- SeatKey activée au premier usage (binding à deviceId hashé).
- SeatKey utilisée depuis un autre deviceId : refus (seat déjà utilisé).
- Release seat par admin en cas de remplacement de poste.

6.2 Endpoints (synthèse)

Domaine	Endpoint	Accès	But
Health	GET /health, GET /ready	public	liveness / readiness
RAG	POST /rag/search	user	retrieval sourcé
RAG	GET /rag/categories	user	catégories dynamiques
Chat	POST/GET /chat/sessions	user	sessions
Chat	POST/GET /chat/messages	user	messages
Ingestion	POST /ingestion/scan	admin	scan
Admin	GET /admin/audit	admin	audit
License	GET /license/status	user	statut
License	POST /admin/seats/release	admin	release

7. Assistant conversationnel (client)

7.1 Pipeline

- Intent Router (LLM local, non-stream) : plan JSON + clarifications.
- Retrieval multi-requêtes : POST /rag/search 1..N.
- Fusion/Diversité/Rerank : dédup + diversité doc/page.
- Answer (LLM local, stream) : réponse structurée + citations numérotées.
- Critic pass (optionnel) : vérifie alignement avec sources.

7.2 UI Sources (user-friendly)

- Citations [1], [2]... cliquables dans le texte.
- Chips de sources sous la réponse (compact) + drawer/flyout 'Sources' à la demande.
- Popup interne avec extrait + bouton 'Ouvrir le PDF' à la page.

8. LLM local (client) — embarqué

- LLM obligatoire ; aucun outil tiers requis.
- Téléchargement modèle automatique (reprise + checksum).
- Lancement en arrière-plan (sans console), logs capturés.
- CPU partout ; GPU automatique si supporté.

9. Sécurité et observabilité

- Rate limiting (429 + Retry-After).
- Audit events admin/chat.
- Logs structurés avec request_id, tenant_id, seat_id/user_id.
- OTel optionnel.

10. Tests et critères d'acceptation

- RAG : chunks pertinents + doc/page/extraits.
- Licences : expiration bloque ; seat binding bloque le double poste ; release admin OK.
- Catégories : ajout/rename/suppression reflétées après scan.
- LLM : machine neuve -> téléchargement modèle -> /v1/models OK -> chat OK.
- Support : support-bundle exporte logs + provisioning redacted + artefacts LLM.